

Note on `locomotion_classifier`

approach + cell map (Synheart Kinematics)

A quick map of the notebook so I can find things fast, then jump into the cells and explore. Cell numbers count every cell from the top (markdown included), as they appear in VS Code. Plain description only — my own findings go at the end.

1 The approach in one paragraph

It detects *how* a person is moving — STATIONARY, ON FOOT, or IN VEHICLE — by the kinematic source of the motion (body, vehicle, or neither). It takes a 5-second window of phone IMU (accel, gyro, linear accel, magnetometer) resampled to 100 points at 20 Hz, turns it into 216 hand-crafted time- and frequency-domain features, feeds those to an XGBoost classifier, then runs the output through three add-on layers — GPS speed fusion, probability smoothing, and a confidence gate — before printing a label. The full flow is drawn as an ASCII diagram in the title cell (**cell 1**); constants `FS=20` and `WINDOW_SIZE=100` are set in **cell 3**.

2 Cell map

Cells 1–3 — setup

Title + architecture diagram (1), pip install of `xgboost/lightgbm` etc. (2), imports, plot style, and the constants `FS`, `WINDOW_SIZE`, class names/colours (3).

Cells 4–6 — “what each class looks like” (§1)

Cell 5 defines `make_window`, which *generates* each class from a seeded RNG: stationary = near-zero + gravity ≈ 9.8 on Z ; on-foot = a 1.7 Hz step bounce + rhythmic gyro; in-vehicle = a 12 Hz aperiodic vibration with a quiet gyro. **Cell 6** plots `accel-X`, `accel-Z`, `gyro-X` for the three.

Cells 7–8 — frequency view (§2)

Cell 8 plots the power spectrum of `accel-Z` per class, marks the dominant frequency, and shades the four bands 0.5–2, 2–4, 4–8, 8–10 Hz. The stated argument: the classes separate most clearly in the spectrum.

Cells 9–11 — the 216 features (§3)

Cell 10 defines `time_features` (11: mean, std, min, max, range, rms, energy, kurtosis, skewness, zero-crossing rate, IQR) and `freq_features` (7: dominant freq, spectral entropy, spectral centroid, 4 band energies); `extract_window` runs both over 12 channels — the 9 per-axis signals plus 3 magnitudes (`np.linalg.norm` of each *xyz* group) — giving $12 \times 18 = 216$. **Cell 11** plots distributions of 8 chosen features over $N = 200$ generated windows per class.

Cells 12–13 — LOSO (§4)

Cell 13 holds the per-user F1 dictionary `loso_results` (8 users scored, 7 skipped for missing classes) and a model-comparison bar (XGBoost 0.818, RF 0.804, LightGBM 0.799, ExtraTrees 0.785, CNN 0.725, MLP 0.692, SVM 0.678). These are fixed numbers typed into the cell, commented “actual results from our training run.”

Cells 14–15 — confusion matrix (§5)

Cell 15 hardcodes a 3×3 matrix (comment: “approximate”). The largest off-diagonal is stationary $\leftrightarrow$ in-vehicle (a still phone reads like a smooth ride).

Cells 16–17 — GPS fusion (§6)

Cell 17 defines a speed $\rightarrow$ weight table (`GPS_SPEEDS`, `GPS_W_STAT/FOOT/VEH`), interpolates it, multiplies the class probabilities by the weights and renormalises; includes a worked example at 0.3 m/s.

Cells 18–19 — inference pipeline (§7)

Four layers: XGBoost $\rightarrow$ GPS fusion $\rightarrow$ averaging the last 5 probability vectors $\rightarrow$ a confidence gate (hold previous state if top prob < 0.45). **Cell 19** runs a generated sequence (10 stationary $\rightarrow$ 15 on-foot $\rightarrow$ 10 vehicle) through it and plots raw vs fused vs final.

Cells 20–22 — live / leftovers

Cell 20 is a stray # `ive_classify.py`; **cell 21** is the Phyphox + iPhone run-instructions markdown (refers to `src/live_classify.py`, which is not in the notebook) plus a next-steps list; **cell 22** is empty.

3 Facts to keep in mind

- **The data is synthetic.** Every signal, spectrum, and feature distribution comes from `make_window` (cell 5); no recordings are loaded.
- **The results are hardcoded.** No dataset, no `xgboost` import, no `.fit()` anywhere. The LOSO, model-comparison, and confusion numbers (cells 13, 15) are written by hand.
- **No orientation test.** 9 of the 12 channels are per-axis and the per-axis `mean` feature is the gravity direction; no rotation/placement check is run.
- **GPS does the stationary-vs-vehicle split** (cells 15–17): the IMU alone has its biggest confusion there.

4 Small bugs

- Cell 19: `prob_buf_sim = deque = []` names a plain list `deque` (`collections.deque` is never imported) and calls `.pop(0)` on it.
- Cell 20: stray empty # `ive_classify.py` (typo of *live_classify*).
- Cell 9 text says “time-domain (11 features)” but the table lists 9 — `min` and `max` are in the code (cell 10) but not the doc.

5 My findings

-
-

-

6 My questions for the meeting

-
-
-